

Universidade de Brasília - UnB
Departamento de Ciência da Computação
Teleinformática e Redes 1

Relatório do Trabalho Final
Simulador de Camada Física e Camada de Enlace

Membros do grupo:

Vinícius Camargo - 202042847
Ivanov Machado dos Santos - 202063669
João Felipe Silva Pereira - 202014388

Brasília - DF
2026

1. Introdução

Este relatório apresenta o desenvolvimento de um simulador didático das camadas de enlace e física, conforme o enunciado do Trabalho Final de Teleinformática e Redes 1. O objetivo do projeto é permitir a visualização e a verificação prática de técnicas de enquadramento, detecção e correção de erros, codificação em banda-base, modulação por portadora, transmissão por socket, recepção em thread e efeito do ruído no canal.

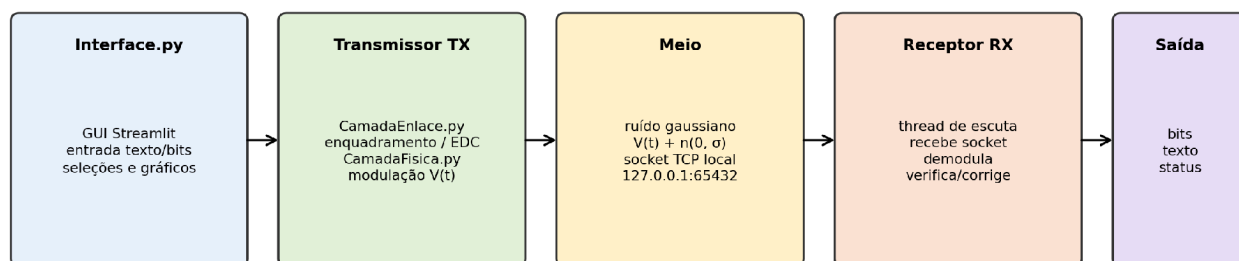
A versão final foi organizada de forma modular em quatro arquivos principais: CamadaFisica.py, CamadaEnlace.py, Interface.py e Simulador.py. A separação permite manter as responsabilidades das camadas bem definidas, mas também integra todo o fluxo transmissor/receptor em uma interface gráfica única.

A interface gráfica foi implementada com Streamlit. Ela permite que o usuário digite texto ou uma sequência binária, selecione métodos de enlace, selecione a técnica da camada física, ajuste o ruído gaussiano do canal e execute a transmissão pelo socket TCP local. O receptor é iniciado em uma thread separada e apresenta os bits demodulados, a validação de enlace e o texto recuperado.

2. Visão geral da arquitetura atual

Conceitualmente, o transmissor prepara os dados na aplicação, converte texto em bits, aplica os métodos de enlace, modula o sinal na camada física e envia o resultado pelo meio de comunicação. O meio soma ruído gaussiano ao sinal e usa socket TCP local para transmitir o pacote. No receptor, uma thread mantém o RX aguardando conexões, recebe o sinal, demodula, aplica a etapa de enlace no sentido de recepção e reconstrói a mensagem.

A arquitetura final segue o fluxo TX/RX solicitado no enunciado: há diferenciação entre transmissor e receptor, interface gráfica, meio de comunicação, ruído adicionado ao sinal e uma rotina principal do simulador coordenando as camadas.



Fluxo implementado: entrada -> TX -> enlace -> física -> ruído -> socket -> RX em thread -> demodulação -> enlace RX -> saída

Figura 1 - Arquitetura atual do simulador modular com TX, RX, socket, thread e ruído.

3. Implementação

3.1 CamadaFisica.py

O arquivo CamadaFisica.py implementa as funções da camada física. Ele contém codificações digitais, modulações por portadora, funções de demodulação e a função de ruído gaussiano. Os sinais são tratados como amplitudes/tensões normalizadas, e o eixo vertical dos gráficos físicos representa $V(t)$ em volts normalizados.

Categoria	Técnica	Descrição resumida
Modulação digital	NRZ-Polar	Mapeia 0 para nível negativo e 1 para nível positivo.
Modulação digital	Manchester	Combina NRZ-Polar com clock usando XOR lógico, gerando transições para sincronismo.
Modulação digital	Bipolar	Representa 0 por zero e alterna os bits 1 entre +1 e -1.
Modulação por portadora	ASK	Altera a amplitude: bit 1 transmite a senoide e bit 0 zera o sinal.
Modulação por portadora	FSK	Altera a frequência: bit 0 usa frequência base e bit 1 usa o dobro da frequência.
Modulação por portadora	QPSK	Agrupa bits de 2 em 2 e mapeia cada par em um ponto complexo da constelação.
Modulação por portadora	16-QAM	Agrupa bits de 4 em 4, usando componentes I e Q com níveis -3, -1, 1 e 3.

A função `gerar_tempo_clock_portadora` cria o eixo de tempo normalizado, o clock de referência e a portadora senoidal base. O tempo utilizado não representa segundos reais; ele representa unidades de tempo da simulação. Em geral, cada bit ocupa uma unidade de tempo, com `AMOSTRAS_POR_BIT` pontos para visualização.

Nas modulações por portadora, o simulador utiliza senoides e sinais complexos. Em ASK, a amplitude é ligada ou desligada conforme o bit. Em FSK, são utilizadas duas frequências normalizadas: frequência base 1 para bit 0 e frequência 2 para bit 1. Em QPSK, cada par de bits é convertido em um símbolo complexo. Em 16-QAM, quatro bits formam um símbolo, sendo dois bits usados para a componente I e dois para a componente Q.

Constelações utilizadas nas modulações por portadora

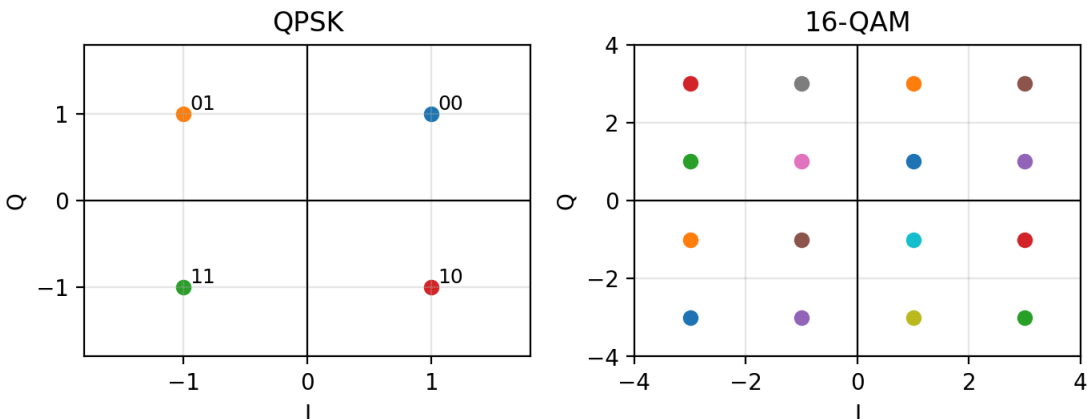


Figura 2 - Constelações utilizadas nas modulações QPSK e 16-QAM.

A demodulação faz o caminho inverso da modulação. NRZ-Polar é demodulado pela média do trecho; Manchester aplica XOR com o clock para recuperar o NRZ; Bipolar usa o módulo da média; ASK usa energia; FSK usa correlação com frequências de referência; QPSK e 16-QAM usam decisão pelo símbolo ou nível mais próximo da constelação.

O ruído gaussiano é acrescentado no canal como $V(t) + n(0, \sigma)$. O parâmetro σ é ajustado na interface e representa a intensidade do ruído em volts normalizados. Quando σ aumenta, o sinal recebido se afasta do sinal transmitido, e a demodulação pode recuperar bits errados, corrompendo o texto final.

3.2 CamadaEnlace.py

O arquivo CamadaEnlace.py implementa os métodos de enquadramento, detecção de erros e correção de erros. A entrada pode ser texto ou bits. Entradas textuais são convertidas para bytes UTF-8 e, em seguida, para uma sequência binária. No transmissor, a camada de enlace prepara os bits que serão enviados à camada física. No receptor, ela remove enquadramentos, verifica códigos de erro e recupera os dados originais quando possível.

Grupo	Método	Finalidade
Enquadramento	Contagem de caracteres	Adiciona um byte inicial com o tamanho total do quadro.
Enquadramento	Flags com inserção de bytes/caracteres	Usa 0x7E como flag e 0x7D como escape para proteger bytes especiais.
Enquadramento	Flags com inserção de bits	Usa a flag 01111110 e insere 0 após cinco bits 1 consecutivos.
Detecção de erros	Paridade par	Adiciona um bit para deixar par a quantidade total de bits 1.
Detecção de erros	Checksum	Calcula soma em complemento de 1 sobre palavras de 16 bits.
Detecção de erros	CRC-32	Calcula um código de redundância cíclica de 32 bits para detectar erros nos dados.
Correção de erros	Hamming	Usa paridades em posições de potência de 2 para detectar e corrigir erro de 1 bit.

Na contagem de caracteres, o primeiro byte do quadro indica o tamanho total. No método com flags e inserção de bytes, a mensagem é delimitada por 0x7E, e bytes especiais dentro dos dados são escapados com 0x7D. No método com inserção de bits, a flag 01111110 delimita o quadro e o transmissor insere um 0 após cinco bits 1 consecutivos para evitar que a flag apareça dentro dos dados.

Para detecção de erros, foram implementados paridade par, checksum e CRC-32. A paridade é simples e detecta alterações em quantidade ímpar de bits. O checksum soma palavras de 16 bits em complemento de 1. O CRC-32 calcula um valor de 32 bits a partir da mensagem original e anexa esse valor ao final dos dados. Na recepção, o CRC é recalculado e comparado com o valor recebido.

Para correção de erros, foi implementado Hamming genérico. O número de bits de paridade é calculado pela condição $2^r \geq k + r + 1$, em que k é a quantidade de bits de dados e r é a quantidade de bits de paridade. Na decodificação, a síndrome indica a posição do erro. Se a síndrome for zero, nenhum erro foi detectado; se for diferente de zero, o bit indicado é invertido para correção.

3.3 Simulador.py

O arquivo Simulador.py implementa a rotina de comunicação entre transmissor e receptor. Ele usa socket TCP local no endereço 127.0.0.1:65432 para representar o meio de comunicação entre TX e RX. O transmissor serializa o pacote de dados, envia o tamanho da mensagem e depois envia o conteúdo pelo socket.

No receptor, uma thread mantém o servidor aguardando transmissões. Quando uma conexão é recebida, o simulador cria uma nova thread para tratar aquele cliente, demodula o sinal recebido usando as funções da CamadaFisica.py e aplica a etapa de recepção da CamadaEnlace.py. O uso de thread evita que a interface gráfica trave enquanto o RX aguarda mensagens.

- HOST utilizado: 127.0.0.1.
- PORTA utilizada: 65432.
- Serialização dos pacotes: pickle, com cabeçalho de tamanho usando struct.
- RX: thread principal de escuta e threads auxiliares para tratar conexões recebidas.

3.4 Interface.py

O arquivo Interface.py implementa a interface gráfica em Streamlit. Ela organiza a tela em blocos de Transmissor TX, Meio de Comunicação e Receptor RX. O usuário escolhe o tipo de entrada, o método de enlace, a técnica de modulação e a intensidade do ruído gaussiano. A interface chama as funções de CamadaEnlace.py, CamadaFisica.py e Simulador.py, sem duplicar a lógica das camadas.

A visualização da camada física mostra quatro gráficos principais: entrada binária do transmissor, clock de referência, sinal limpo comparado com o sinal recebido com ruído e bits recuperados após demodulação. Todos os gráficos estão em função do tempo normalizado. O eixo y representa bit lógico, amplitude de clock ou tensão/amplitude $V(t)$ em volts normalizados, conforme o gráfico.

Comando principal de execução:

```
py -m streamlit run Interface.py
```

4. Decisões de projeto

- A interface foi separada em Interface.py, enquanto Simulador.py ficou responsável por socket, thread e rotina TX/RX.
- O socket TCP local foi escolhido para demonstrar comunicação real entre transmissor e receptor sem exigir duas máquinas físicas.
- O receptor roda em thread para permanecer aguardando transmissões sem bloquear a interface Streamlit.
- O ruído foi modelado como variável gaussiana $n(0, \sigma)$, somada ao sinal modulado no meio de comunicação.
- O tempo dos gráficos é normalizado, pois o objetivo é visualizar relações entre bits, clock, sinal e recuperação, não medir uma taxa física real em segundos.
- Os sinais são tratados como tensões/amplitudes normalizadas em Volts. Não foi feita modelagem separada de potência em watts.

5. Membros e atividades

Membro	Atividades desenvolvidas
Vinícius Camargo	Implementação e explicação das modulações digitais NRZ-Polar, Manchester e Bipolar, além das modulações por portadora ASK e FSK. Também participou dos testes de execução da camada física, validação dos gráficos, integração com socket/thread e ajustes da interface.
Ivanov Machado	Implementação e explicação das modulações por portadora QPSK e 16-QAM. Também ficou responsável pelos métodos de enquadramento da camada de enlace: contagem de caracteres, enquadramento com flags e inserção de bytes/caracteres, e enquadramento com flags e inserção de bits.
João Felipe	Implementação e explicação dos métodos de detecção e correção de erros da camada de enlace, incluindo bit de paridade par, checksum, CRC-32 e Hamming. Também participou da validação dos resultados de verificação, correção e recuperação das mensagens.

6. Conclusão

O desenvolvimento do simulador permitiu relacionar os conceitos teóricos de redes com uma implementação prática. Na camada de enlace, foi possível observar como os quadros são delimitados e como mecanismos de detecção e correção de erros aumentam a confiabilidade da transmissão. Na camada física, foi possível visualizar como a mesma sequência de bits assume formas diferentes dependendo da técnica de codificação ou modulação escolhida.

A versão final também incorpora a arquitetura de transmissor e receptor, usando socket TCP local e thread para representar a comunicação entre processos. O meio de comunicação passou a incluir ruído gaussiano ajustável, permitindo demonstrar o impacto do canal na recuperação dos bits. Como resultado, o grupo obteve um simulador funcional, modular e mais próximo do diagrama conceitual do trabalho.

7. Referências

1. Enunciado do Trabalho Final de Teleinformática e Redes 1, Departamento de Ciência da Computação, Universidade de Brasília.
2. Material de aula da disciplina sobre camada física, modulação, enquadramento e controle de erros.
3. Documentação oficial das bibliotecas Python utilizadas: Streamlit, NumPy, Matplotlib e biblioteca padrão socket/threading.